Mitigating False Completion in Vision-Language-Action Models

Anonymous Authors

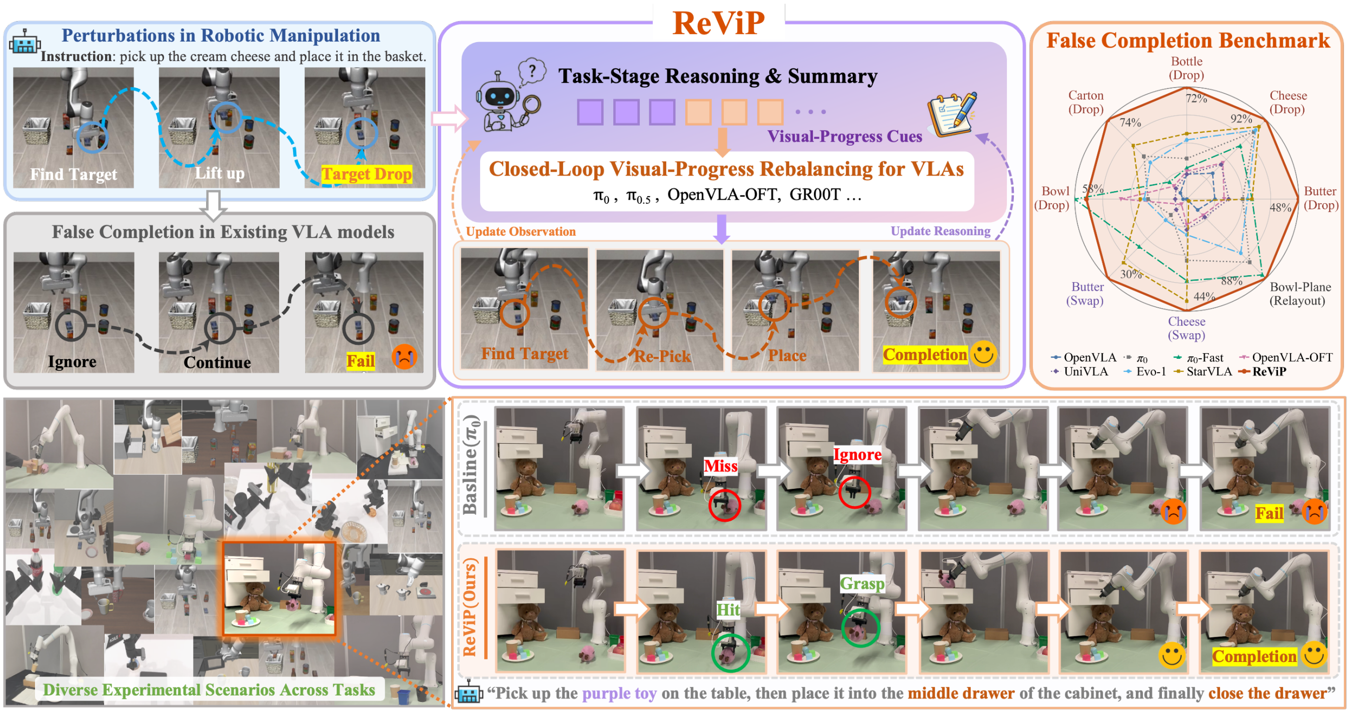

Perturbations in Robotic Manipulation
Instruction: pick up the cream cheese and place it in the basket.
Find Target
Lift up
Target Drop
False Completion in Existing VLA models
Ignore
Continue
Fail
ReViP
Task-Stage Reasoning & Summary
Visual-Progress Cues
Closed-Loop Visual-Progress Rebalancing for VLAs
π_0, $\pi_{0.5}$, OpenVLA-OFT, GR00T …
Update Observation
Update Reasoning
Find Target
Re-Pick
Place
Completion
False Completion Benchmark
Bottle (Drop)
Cheese (Drop)
Butter (Drop)
Bowl-Plane (Relayout)
Cheese (Swap)
Butter (Swap)
Bowl (Drop)
Carton (Drop)
72%
92%
48%
88%
44%
30%
58%
74%
OpenVLA
π_0
π_0-Fast
OpenVLA-OFT
UniVLA
Evo-1
StarVLA
ReViP
Diverse Experimental Scenarios Across Tasks
Basline(π_0)
Miss
Ignore
Fail
ReViP(Ours)
Hit
Grasp
Completion
"Pick up the purple toy on the table, then place it into the middle drawer of the cabinet, and finally close the drawer"

Fig. 1. **From false completion to closed-loop visual-progress rebalancing. Top (left):** when an object drops during online execution, existing VLAs may ignore the visible conflict and act as if the task were complete, *i.e*, *false completion*. **Top (middle):** ReViP closes the perception-action loop: it performs task-stage reasoning and yields visual-progress cues to ground representations and rebalance action generation across heterogeneous VLAs, enabling robots to detect, re-pick, and complete the task. **Top (right):** the first benchmark dedicated to false completion covers *Object Drop*, *Distractor Swap*, and *Relayout* across eight tasks to systematically expose controlled visual-progress conflicts. **Bottom:** across extensive simulation and real-world scenarios, ReViP improves false-completion recovery and task success over strong VLA and VLA+VLM baselines.

***Abstract*—Although Vision-Language-Action models (VLAs) have recently shown promise in robotic manipulation, they may continue according to presumed task progress even when visual observations clearly indicate otherwise, revealing a breakdown in the perception-action loop. In this work, we term this counter-intuitive failure under visual-progress conflicts *false completion*, a critical issue hindering reliable robotic deployment. However, this problem remains underexplored in prior work, leaving two key challenges: 1) understanding why imitation-trained VLAs fail to respond adequately to visual evidence of task progress, and 2) enabling such evidence to guide ongoing execution through the policy's action-generation process. To investigate this gap, we introduce the *False-Completion Benchmark Suite*, comprising diverse tasks across three controlled settings: *Object Drop*, *Distractor Swap*, and *Relayout*. Our data-to-model diagnostics further reveal a *state-dominant bias in progress estimation*, reflecting imbalanced task-progress grounding under imitation learning. To address this problem, we propose ReViP, a closed-loop visual-progress rebalancing framework that internalizes visual progress verification into action generation. Specifically, *Visual-Progress Grounding* grounds task progress in current visual evidence through task-stage reasoning and integrates the resulting cues into the policy representation through lightweight feature modulation. *Stage-Aware Attention Rebalancing* combines these cues with execution context to adaptively rebalance action-generating attention through group-wise dual calibration, thereby closing the loop between visual progress and action generation. Moreover, we introduce ReViP-Flash, an efficient variant that distills the external observer into a lightweight in-policy module while retaining the robustness of ReViP. Finally, extensive simulation and real-world experiments show that ReViP consistently mitigates false completion and achieves state-of-the-art task success compared with strong VLA and VLA+VLM baselines, while remaining plug-and-play across heterogeneous VLA backbones with less than 0.4% additional trainable parameters.**

***Index Terms*—Vision-language-action models, closed-loop framework, vision-language models, robot learning.**

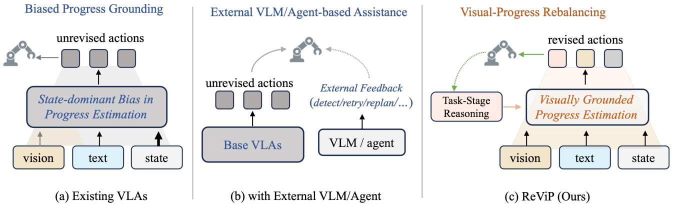
I. INTRODUCTION

RECENTLY, Vision-Language-Action models (VLAs) [1]–[8] have demonstrated promising robotic manipulation capabilities by extending pretrained vision-language models (VLMs) toward executable action generation. However, reliable deployment requires more than conditioning action generation on updated observations: the generated actions must also respond to task-relevant visual changes, particularly when these changes invalidate presumed task progress.

In practice, existing methods [2], [4], [9], [10] may fail to respond to *visual-progress conflicts*, where current visual evidence contradicts their presumed task progress. As shown in Figure 1 (top left), after a grasped object unexpectedly drops, policies may continue toward the destination and execute the placement motion as if the object were still held. We term this counterintuitive failure "false completion", *i.e*, a policy continues or terminates execution according to the presumed task progress that is no longer supported by visual evidence, revealing a breakdown in the perception-action loop. False completion undermines the reliability of robotic manipulation. However, few prior works have directly explored this issue, leaving two key challenges: **(1)** understanding why imitation-trained VLAs fail to respond adequately to visual evidence of task progress, and **(2)** enabling current visual evidence to guide ongoing execution through the policy's action-generation process.

To address the first challenge, we introduce **False-Completion Benchmark Suite** and conduct data-to-model diagnostics to understand why imitation-trained models underutilize visual evidence of task progress. Built upon LIBERO [11], our benchmark suite is the first dedicated to evaluating VLA behavior under controlled visual-progress conflicts (top right in Figure 1). It comprises eight tasks spanning three complementary dimensions: *Object Drop* invalidates object possession, *Distractor Swap* challenges target identity, and *Relayout* breaks demonstration-specific spatial assumptions. Combining evaluations on this benchmark with real-world studies, we then trace false completion through recovery-data augmentation and state-masking experiments. These diagnostics suggest a *state-dominant bias in progress estimation*: current imitation-trained VLAs (*e.g.*, π_0 [9] and OpenVLA-OFT [12]) may over-rely on execution-state cues, acting on unsupported progress assumptions rather than revise their actions in response to the conflicting visual evidence, as shown in Figure 2 (a). Building on this diagnosis, an intuitive solution is to introduce external planning, failure detection, and recovery guidance, as exemplified by existing state-of-the-art VLA+VLM and agentic techniques (*e.g.*, VoLoAgent [13] and HarnessVLA [14]). However, as illustrated in Figure 2 (b), corrective manipulation still depends on the base VLA, which may remain constrained by incorrect assumptions about task progress. Such external assistance can facilitate recovery but does not directly address how the policy interprets stage progress.

To address the second challenge and overcome these limitations, we propose **ReViP**, a closed-loop **Vi**sual-**P**rogress **Re**balancing framework that internalizes visual stage verification into action generation. Motivated by the diagnosed state-dominant bias, our core insight is that task progress must remain grounded in current visual evidence, allowing the policy to revise its actions when visual observations contradict prior execution assumptions As illustrated in Figure 1 (top middle), ReViP realizes this insight through two complementary mechanisms. Specifically, *Visual-Progress Grounding* employs a Task-Stage Observer to perform task-stage reasoning, verifying stage conditions against current visual evidence and integrating the resulting visual-progress cues into policy representations through lightweight feature modulation. *Stage-Aware Attention Rebalancing* then combines these cues with execution context to adaptively rebalance action-generating attention. Through group-wise dual calibration, it controls the strength and token-wise allocation of attention modulation, emphasizing stage-relevant visual evidence. As shown in Figure 2 (c), these mechanisms connect visual stage verification to corrective action generation, closing the loop between visual progress and ongoing execution.


Biased Progress Grounding
unrevised actions
State-dominant Bias in Progress Estimation
vision
text
state
External VLM/Agent-based Assistance
unrevised actions
External Feedback (detect/retry/replan/...)
Base VLAs
VLM / agent
Visual-Progress Rebalancing
revised actions
Task-Stage Reasoning
Visually Grounded Progress Estimation
vision
text
state

(a) Existing VLAs (b) with External VLM/Agent (c) ReViP (Ours)

Fig. 2. (a) Existing VLAs may suffer from state-dominant bias in progress estimation; (b) External VLMs/agents organize recovery, while fine-grained manipulation still depends on base VLA's internal responsiveness to visual evidence; (c) ReViP internalizes visually grounded task progress into action generation to revise policy actions in response to visual evidence.

For efficient deployment, we further develop ReViP-Flash, which distills the external observer into a lightweight in-policy module via an event-aware Q-Former, eliminating external VLM inference. To the best of our knowledge, we present the first systematic study of *false completion* in VLAs under visual-progress conflicts, combining a dedicated benchmark, data-to-model diagnostics, and a visual-progress rebalancing framework.

Finally, extensive evaluations on the False-Completion Benchmark, single-arm LIBERO [11] and challenging LIBERO-Plus [15], bimanual RoboTwin 2.0 [16], and real-world manipulation demonstrate the effectiveness of ReViP (bottom in Figure 1). ReViP significantly mitigates false completions under unseen visual-progress conflicts and achieves SOTA task success rates over strong VLA and VLA+VLM baselines. Importantly, ReViP is plug-and-play across heterogeneous VLA backbones and observers, introducing less than 0.4% additional trainable parameters and only 17ms additional latency under our asynchronous reasoning setup. Moreover, combining ReViP with recovery-data augmentation further improves both recovery and standard manipulation, demonstrating the complementary benefits of richer recovery supervision and visually grounded action revision (Sec. V). Our contributions are summarized as follows:

- We characterize false completion and introduce the False-Completion Benchmark Suite with dedicated evaluation metrics. Our data-to-model studies combine benchmark evaluations with real-world diagnostics to reveal a state-dominant bias in progress estimation, helping explain why imitation-trained VLAs underutilize conflicting visual evidence.

- We propose ReViP, which internalizes visual stage verification into action generation through Visual-Progress Grounding and Stage-Aware Attention Rebalancing, enabling verified progress evidence to guide corrective actions. We further develop ReViP-Flash, which distills the external observer into a lightweight in-policy module for efficient deployment.
- Extensive simulation and real-world experiments demonstrate that ReViP mitigates false completion and achieves state-of-the-art task success over strong VLA and VLA+VLM baselines. It remains plug-and-play across heterogeneous VLA backbones with less than 0.4% additional trainable parameters and complements recovery-data augmentation to improve both recovery and standard manipulation.

The remainder of this paper is organized as follows. Section II reviews related work. Section III introduces the False-Completion Benchmark Suite and presents data-to-model diagnostics. Section IV details ReViP and its efficient variant, ReViP-Flash. Section V presents simulation and real-world evaluations, ablation studies, and efficiency analysis. Finally, Section VI concludes the paper.

II. RELATED WORK

A. Vision-Language-Action Models

VLA models [1], [4], [5], [9], [17]–[20] connect perception to control by mapping pretrained vision-language representations to executable actions. Early approaches in the RT line (RT-1 [21], RT-2 [1], and RT-H [22]) explored transformer-based action generation and language-conditioned control. Large-scale datasets such as Open X-Embodiment [23] and DROID [24] further support the development of generalist robotic policies. Recently, the π series (FAST [4], π_0 [9], and $\pi_{0.5}$ [6]) proposed heterogeneous co-training across robots, generating continuous actions with flow-matching decoders [25] or efficient action tokenization. Beyond these, a range of VLA variants [3], [5], [26]–[30] pursue stronger robot manipulation through structured reasoning, spatial grounding, and efficient architectures. For example, CoT-VLA [26] integrates visual chain-of-thought reasoning that predicts subgoal frames before generating actions, and the GR00T series [10] targets generalist humanoid robots by developing foundation VLAs for complex whole-body tasks.

Despite these advances, visual verification of task progress and action revision under conflicting visual evidence remains insufficiently explored. In contrast, we study this missing progress-grounding capability and the resulting phenomenon of *false completion*.

B. External VLMs and Agentic Feedback for VLAs

Directly fine-tuning the VLM inside a VLA on robot data has been reported to degrade its pretrained multimodal understanding because of distribution shift and forgetting [6], [9], [31], which motivates keeping a strong external VLM for task understanding [32], [33]. Recent methods fall into two patterns: planning-based guidance and evaluation-based feedback. The former decomposes long-horizon instructions into actionable subgoals [34]–[36], while the latter assesses execution outcomes and provides failure feedback to support recovery [37], [38].

More recent agentic systems further extend this paradigm by treating VLAs as manipulation primitives, combining planning and feedback to coordinate VLA primitives through replanning, re-staging, and retries [13], [14]. Frameworks such as VLATest [39] use auxiliary evaluators to probe and stress-test VLA behavior across diverse variations.

However, for false completion, the missing capability goes beyond external failure detection or agentic recovery. As illustrated in Figure 2(b), existing VLM- or agent-assisted systems primarily intervene through external supervision, recovery, or backtracking, without directly addressing how task progress is grounded within the base VLA. For example, CycleVLA [40] and HarnessVLA [14] primarily organize recovery through explicit execution decisions, while corrective manipulation remains constrained by the underlying policy's use of task-progress evidence.

In contrast, our proposed ReViP internalizes task-stage evidence verified by a VLM-based observer into policy representations and rebalances visual and execution cues within action-generating attention.

C. Progress and Multimodal Modeling in Robotic Policies

Recently, a few studies have begun to examine task progress [41], [42] or multimodal interaction [43]–[45] in robotic policies. For instance, ProgressVLA [41] uses scalar progress signals to improve temporal awareness and long-horizon action generation. However, such coarse estimates provide limited information about the concrete task stage, making them insufficient to determine whether presumed progress remains visually supported after unexpected scene changes.

Another line of work studies multimodal inputs through input removal, optimization regularization, or token redesign. State-free Policy [43] removes proprioceptive inputs to reduce shortcut learning, GAP [44] regularizes phase-dependent vision-proprioception optimization in single-task diffusion policies, while ThinkProprio [45] strengthens proprioceptive modeling for action generation. While these studies advance progress estimation and multimodal modeling, how policies respond when visual evidence contradicts presumed task progress remains insufficiently explored, particularly when they fail to revise ongoing execution.

In this paper, we investigate false completion through a dedicated benchmark and data-to-model diagnostics, and develop ReViP to ground task progress in visually verified stage evidence and rebalance multimodal cues during action generation.

III. UNDERSTANDING FALSE COMPLETION IN VLAS

In this section, we study false completion from phenomenon to mechanism: formalizing the failure (Sec. III-A), introducing the False-Completion Benchmark Suite with evaluation metrics (Sec. III-B), and conducting data-to-model diagnostics (Sec. III-C).

A. Formulating False Completion

Here, we formalize false completion as a failure in which a policy advances or terminates execution based on presumed task progress that is contradicted by current visual evidence. At time step t, the visual observation I_t consists of synchronized images from a camera set $\mathcal{C}$, including egocentric and exocentric views:

$$I_t = \left\{ I_t^{(c)} \in \mathbb{R}^{H_c \times W_c \times 3} \;\middle|\; c \in \mathcal{C} \right\}. \tag{1}$$

Given I_t, the proprioceptive state S_t, and the language instruction ℓ, a VLA policy π_θ an action chunk of length n:

$$A_t = \pi_\theta\big[a_t, a_{t+1}, \dots, a_{t+n-1} \mid (I_t, S_t, \ell)\big]. \tag{2}$$

Although policy receives updated observations at each replanning step, reliable execution requires action revision when task-relevant conditions change.

Let $\hat{p}_t \in \{0,1\}$ denote the policy's *implicit progress estimate* for the currently relevant task stage, where $\hat{p}_t = 1$ indicates that the policy behaves as if the corresponding stage condition is satisfied. This is a conceptual description of behavior, not an explicit policy output or a measured internal state. Let $G_t(I_t, \ell) \in \{0,1\}$ denote a task-conditioned visual predicate indicating whether the same condition is supported by current observations.

Visual-Progress Conflict. A *visual-progress conflict* occurs when presumed progress contradicts visual evidence:

$$\hat{p}_t = 1 \;\wedge\; G_t(I_t, \ell) = 0. \tag{3}$$

For example, existing VLAs such as π_0 [9] may continue transporting toward the goal after an object drops, behaving as if it were still held, as illustrated in Figure 1. This conflict calls for revising the presumed progress and generating corrective actions.

False Completion. A closed-loop policy may recognize the discrepancy and revise its subsequent actions. Thus, *False completion* arises when it instead advances or terminates execution according to the invalid progress assumption. At the terminal stage, $\hat{p}_t = 1$ denotes presumed task completion, and $G(I_t, \ell)$ is the terminal goal predicate. The corresponding completion outcomes are:

$$\begin{aligned} \text{True Completion} &= \big[\hat{p}_t = 1 \;\wedge\; G(I_t, \ell) = 1\big], \\ \text{False Completion} &= \big[\hat{p}_t = 1 \;\wedge\; G(I_t, \ell) = 0\big]. \end{aligned} \tag{4}$$

Therefore, false completion concerns how the policy acts on conflicting evidence, rather than the mere presence of a conflict or an unsuccessful rollout, motivating our dedicated benchmark suite introduced below.

B. False-Completion Benchmark Suite

To evaluate this behavior, we introduce the **False-Completion Benchmark Suite**, built upon LIBERO [11]. Unlike existing manipulation and robustness benchmarks [11], [15], [16], [46] that primarily measure final task success, our suite introduces controlled interventions to invalidate task-progress assumptions and evaluate the policy's response.

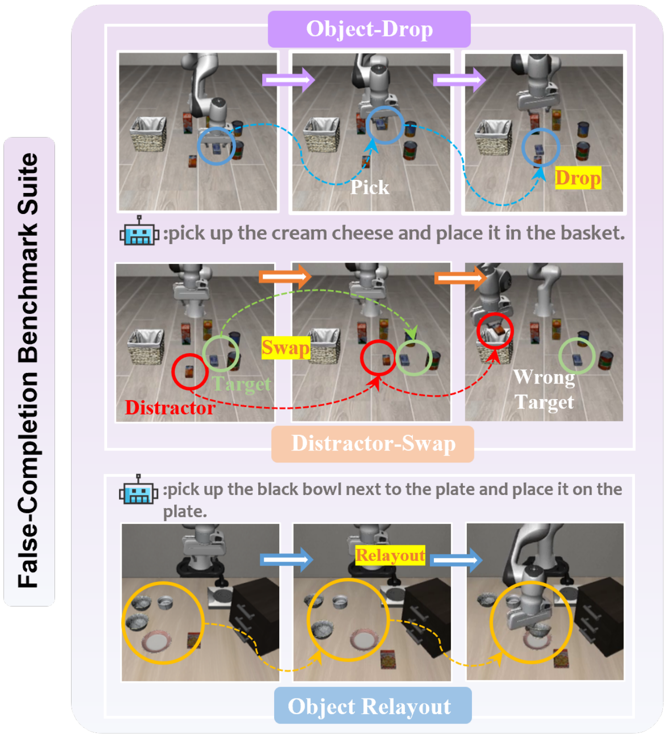

False-Completion Benchmark Suite
Object-Drop
Pick
Drop
:pick up the cream cheese and place it in the basket.
Swap
Target
Distractor
Wrong Target
Distractor-Swap
:pick up the black bowl next to the plate and place it on the plate.
Relayout
Object Relayout

Fig. 3. Illustrative examples of our False Completion Benchmark. The benchmark consists of three complementary perturbation sources, *Object Drop*, *Distractor Swap*, and *Relayout*, which induce conflicts in object possession, target identity, and spatial progress, respectively.

The benchmark contains eight manipulation tasks organized into three distinct conflict dimensions: *Object Drop*, *Distractor Swap*, and *Relayout*. As shown in Figure 3, they instantiate complementary forms of visual-progress conflicts: drops invalidate object possession, swaps challenge target identity, and relayout breaks demonstration-specific spatial assumptions. Specifically, the language instruction remains unchanged, while the environment is perturbed either at initialization or during execution. By invalidating demonstration-specific execution assumptions, these interventions test whether policies can adapt beyond replaying familiar trajectories. Instead, the policy must re-ground its presumed task progress in the current scene and revise its subsequent actions when that progress is invalidated. Below, we will describe the perturbation sources.

Object Drop. This setting includes *Small Object Drop*, *Large Object Drop*, and *Bowl Drop in Cabinet* covering different object scales and contact contexts. After the object has been successfully grasped, a controlled release causes it to drop during subsequent execution. A successful policy must recognize that object possession has been invalidated, return to the object, and recover rather than continuing the nominal placement trajectory.

Distractor Swap. This family evaluates whether a policy maintains the correct target identity when the target and a visually similar distractor exchange positions. Their initial poses are swapped while the language instruction remains fixed. A successful policy must therefore use current visual evidence rather than replaying a demonstration-specific trajectory toward the original target location.

Relayout. This family disrupts demonstration-specific spatial assumptions while preserving the language goal. The target object and its goal region are jointly relocated to new feasible positions, requiring the policy to re-ground its spatial reasoning in the current scene and generate actions accordingly.

These perturbations provide a controlled test of whether a VLA revises its actions under visual-progress conflicts rather than continuing along an invalid execution trajectory. Perturbation triggers are deterministic (*e.g.*, reproducible drop rule) for fair comparison. Full benchmark-construction details are provided in the supplementary material.

False-Completion Success Rate. Final task success alone does not isolate recovery ability, as rollouts may fail before the visual-progress conflict is triggered. We therefore complement task success rate (SR) with *False-Completion Success Rate (FCSR)*, computed over trials that validly reach the designated conflict:

$$\mathrm{FCSR} = \frac{1}{|\mathcal{V}|}\sum_{i\in\mathcal{V}} G(I_{T_i}^{(i)}, \ell_i), \tag{5}$$

where T_i denotes the terminal step of trial i, and $\mathcal{V}$ denotes the set of trials satisfying task-specific conflict-validity criteria.

A trial is included in $\mathcal{V}$ only when the intended perturbation is successfully instantiated and the rollout reaches the stage at which the policy must respond to it. Trials that fail earlier due to unrelated manipulation errors are excluded. For *Object Drop*, the object must first be grasped and the scripted release must occur as intended. For *Distractor Swap* and *Relayout*, all successfully initialized perturbed trials are included. Full Details are provided in Sec. III of the supplementary material. FCSR therefore measures end-to-end recovery following a valid visual-progress conflict, jointly reflecting conflict recognition and corrective execution.

By excluding failures before the designated conflict, the benchmark and FCSR provide a controlled setting for investigating whether and how VLAs revise execution in response to contradictory visual evidence.

C. Understanding False Completion from Data to Model

Building on the controlled diagnostic setting, we next trace false completion from data to model to understand *why current VLAs struggle to revise their presumed task progress under visual-progress conflicts.* We explore this issue from two perspectives. An intuitive solution to false completion is to augment VLAs with additional failure-recovery trajectories that demonstrate how to respond when task progress is disrupted. At the model level, proprioceptive cues often correlate with task advancement in successful demonstrations, potentially encouraging policies to rely on these cues even when current visual evidence contradicts the presumed progress.

We therefore examine two diagnostic questions: **Q1:** *Can additional recovery augmentation effectively mitigate false completion?* and **Q2:** *Does over-reliance on proprioceptive execution cues limit the policy's response to conflicting visual evidence?* To answer these, we conduct recovery-data supervision and modality-controlled experiments in simulation and the real world.

TABLE I

Recovery-data augmentation exposes a trade-off. Adding more recovery trajectories improves π_0's false-completion recovery (FCSR: 36% → 63%) but reduces standard LIBERO performance (SR: 94.2% → 74.6%).

Methods	**Added Recovery Traj.**	**Avg. FCSR↑**	**Standard LIBERO Avg. SR↑**
π_0 [9]	0	36%	94.2%
π_0 + recovery aug.	+80 total (10 per task)	45%	86.3%
π_0 + recovery aug.	+160 total (20 per task)	54%	81.5%
π_0 + recovery aug.	+240 total (30 per task)	63%	74.6%

TABLE II

Real-world modality-controlled behavior. #Proceed-to-Goal and #Back-to-Object count policy responses to a state-vision mismatch over 50 trials, with SR measures standard rollout.

Modality	**#Proceed-to-Goal ↓**	**#Back-to-Object ↑**	**Rollout SR ↑**
State-Enabled [9]	46 / 50	4 / 50	**70%**
State-Masked [43]	**17 / 50**	**33 / 50**	40%

(1) Recovery augmentation mitigates false completion but at data-collection and standard-performance costs. To answer Q1, we augment the standard training set of the π_0 baseline [9] with three sets of 80, 160, and 240 Object-Drop-style recovery trajectories, with the largest requiring approximately nine hours of 3D-mouse teleoperation to collect. As shown in Table I, scaling the recovery data from 0 to 240 demonstrations improves π_0's success rate under false-completion perturbations from 36% to 63%, but reduces its standard LIBERO performance from 94.2% to 74.6%.

Thus, recovery behavior is learnable from additional demonstrations, but recovery augmentation alone introduces substantial collection cost and an increasing degradation of standard manipulation. More importantly, additional recovery trajectories demonstrate how to recover, but do not explicitly supervise the verification of whether presumed task progress remains valid under current visual evidence.

TABLE III

Modality-controlled experiments on standard LIBERO. Success rates (%) are reported.

Methods	**Spatial ↑**	**Object ↑**	**Goal ↑**	**10 ↑**	**Average ↑**
π_0 [9]	96.8	98.8	95.8	85.2	94.2
w/ State-Masked [43]	94.8 (↓ 2.0)	95.8 (↓ 3.0)	90.6 (↓ 5.2)	80.6 (↓ 4.6)	90.5 (↓ 3.7)

(2) State masking improves responsiveness to visual conflicts but degrades basic manipulation. To answer Q2, we reproduce a controlled visual-progress conflict in a real-world *cup pick-and-place* task (detailed in Sec V-H). Specifically, we close the gripper before it reaches the cup, creating a mismatch between grasp-like proprioceptive cues and visual evidence that the cup remains on the table. Across 50 experimental trials, we then record whether the robot continues toward the goal (*Proceed-to-Goal*) or returns to the object (*Back-to-Object*). To isolate the role of proprioception, we further implement a state-free variant of the π_0 baseline by zeroing its proprioceptive input (*state masking*) [43], and evaluate it under the same conflict. As shown in Table II, the vanilla policy with state enabled (*i.e*, π_0) proceeds toward the goal in 46 of 50 trials and returns to the object in only 4, despite clear visual evidence that the grasp has failed. State masking markedly shifts this behavior, reducing *Proceed-*

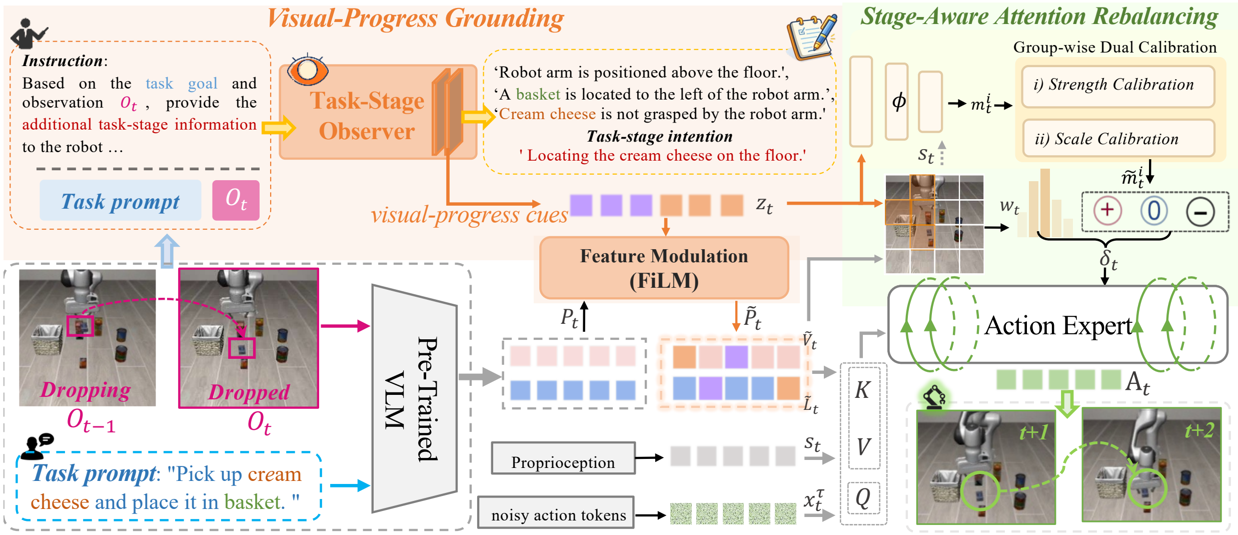

Visual-Progress Grounding
Instruction: Based on the task goal and observation O_t, provide the additional task-stage information to the robot …
Task prompt
O_t
Task-Stage Observer
'Robot arm is positioned above the floor.',
'A basket is located to the left of the robot arm.',
'Cream cheese is not grasped by the robot arm.'
Task-stage intention
' Locating the cream cheese on the floor.'
visual-progress cues
z_t
Feature Modulation (FiLM)
P_t
$\tilde{P}_t$
Dropping
O_{t-1}
Dropped
O_t
Pre-Trained VLM
Task prompt: "Pick up cream cheese and place it in basket. "
$\tilde{V}_t$
$\tilde{L}_t$
Proprioception
s_t
noisy action tokens
x_t^{τ}
K
V
Q
Stage-Aware Attention Rebalancing
Group-wise Dual Calibration
ϕ
m_t^i
s_t
i) Strength Calibration
ii) Scale Calibration
$\tilde{m}_t^i$
w_t
+
0
−
δ_t
Action Expert
A_t
t+1
t+2

Fig. 4. **Overview of ReViP framework.** First, *Visual-Progress Grounding* verifies stage conditions through task-stage reasoning to produce visual-progress cues z_t, which modulate P_t via FiLM to obtain grounded context tokens $\widetilde{P}_t$. Second, Stage-Aware Attention Rebalancing predicts group-wise modulation signals m_t^i and calibrates their strength and token-wise allocation to obtain δ_t, which rebalances the action expert's attention to guide action chunk generation A_t. New observations refresh the progress cues, closing the loop between stage verification and action generation.

to-Goal responses (46/50→17/50) and increasing *Back-to-Object* responses (4/50→33/50). This shift suggests that current observations already contain visual evidence sufficient to support correction, but that this evidence can be overridden by execution-consistent proprioceptive cues. However, simply removing state is not a viable solution: it also reduces standard task success on the real robot (70%→40%) and LIBERO (94.2%→90.5%), with degradation across all suites (Table III).

These results suggest that imitation-trained VLAs benefit from proprioceptive cues for standard manipulation, yet may over-rely on them to infer task progress and continue execution despite conflicting visual evidence.

(3) False completion reflects a state-dominant bias in progress estimation. Recovery augmentation shows that corrective behaviors can be learned, while state masking reveals that execution-state cues influence whether the policy responds to conflicting visual evidence. In our experiments, both interventions improve conflict recovery or corrective responses at the cost of standard task performance. These findings motivate examining how imitation-trained VLAs use visual and execution cues to infer task progress, beyond the availability of recovery demonstrations or proprioceptive inputs alone.

In successful demonstrations, proprioceptive transitions are naturally correlated with task advancement, making execution cues a convenient proxy for progress. Behavior cloning can encourage reliance on this correlation through action supervision, without explicitly requiring the policy to verify task-stage conditions against current visual evidence. This can favor a **state-dominant bias in progress estimation**: execution cues sustain presumed progress even when the observed scene contradicts it. The resulting behavior is consistent with our state-masking findings, where removing proprioceptive inputs increases corrective responses but compromises standard manipulation. Thus, receiving updated observations does not ensure that visual evidence effectively revises progress-dependent actions, leaving the perception-action loop incomplete.

Crucially, this diagnosis leads to the second challenge identified in the Introduction: enabling visual evidence of task progress to guide ongoing execution through the policy's action-generation process. Addressing this challenge requires more than progress estimation: the policy must continually ground its presumed progress in current visual observations and use this grounding to revise subsequent actions. This insight directly motivates ReViP, our framework for *closed-loop visual-progress rebalancing*. Further detailed analysis is provided in the supplementary material.

IV. ReViP

In this section, we present the overall framework of ReViP (Sec. IV-A) and detail the implementation of Visual-Progress Grounding (Sec. IV-B) and Stage-Aware Attention Rebalancing (Sec. IV-C).

A. *Overview*

As illustrated in Figure 4, we present the ReViP framework, built upon base VLAs comprising a VLM and an action expert. The VLM encodes observations I_t and instructions ℓ into vision-language context tokens P_t, while the action expert generates an action chunk A_t conditioned on P_t and proprioceptive state S_t.

The main idea behind ReViP is to ground task progress in current visual evidence and use it to guide action generation through two core components.

First, Visual-Progress Grounding performs task-stage reasoning through a task-stage observer to verify stage conditions and encode the resulting evidence into visual-progress cues z_t. Then, the cues modulate P_t through lightweight feature modulation, yielding a progress-aware representation $\widetilde{P}_t$. Second, Stage-Aware Attention Rebalancing predicts group-wise modulation signals from z_t and execution state, then calibrates their strength and token-wise allocation to obtain δ_t. These

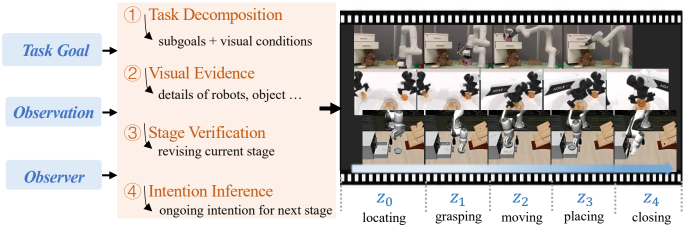


Fig. 5. **Four-step task-stage reasoning.** The observer decomposes the task, extracts task-relevant visual evidence, verifies stage transitions, and infers the immediate objective to produce visual-progress cues.

calibrated signals dynamically rebalance the action expert's attention over the progress-aware context and execution inputs, guiding subsequent action generation. We also develop ReViP-Flash, which distills the external observer into an in-policy module for efficient deployment. Finally, during execution, ReViP updates the stage cues from new observations, closing the loop between visual progress verification and action generation.

B. Visual-Progress Grounding

Visual-Progress Grounding aims to provide the policy with task-stage evidence verified against current observations. To obtain this evidence, we perform task-stage reasoning through a frozen external VLM, termed the *task-stage observer*.

Given the task instruction ℓ and current observation I_t, this four-step reasoning process verifies stage conditions and infers the immediate objective, as described in Figure 5. Specifically, ① *task decomposition* parses the task goal into ordered stage-level subgoals:

$$\mathcal{G}_\ell = \{(g_k, c_k)\}_{k=1}^{K}, \tag{6}$$

where g_k denotes the stage objective and c_k its visual completion conditions. ② *visual-evidence extraction* identifies evidence e_t relevant to these conditions, including the visible robot configuration, target-object status, and spatial relations to the goal. ③ e_t is then evaluated against $c_k{}_{k=1}^{K}$ through *stage verification* to infer the currently valid stage $\hat{k}_t$. Importantly, this estimate is not constrained to advance monotonically: if previously satisfied conditions become invalid, the observer revises the stage accordingly. ④ based on the verified stage, *intention inference* determines the immediate objective r_t for advancing execution or returning to a previous stage. For example, when the target *cream cheese* accidentally falls while the robot is transporting it toward the basket, the observer analyzes (I_t, ℓ) and identifies critical evidence such as *"Cream cheese is not grasped by the robot arm"*. This visual evidence invalidates the presumed transport stage and updates the immediate objective to *"Locating the cream cheese on the floor"*, as shown in Figure 4.

To make the reasoning results available to the policy, we pool and project the observer's reasoning features into a compact visual-progress cue, following prior work on LLM embeddings [47]–[49]:

$$z_t = \mathrm{Proj}(\mathrm{Pool}(f_t)) \in \mathbb{R}^d,\ \ f_t = \mathrm{Observer}_L\left(I_t, \ell, \mathcal{G}_\ell\right), \tag{7}$$

where f_t denotes the selected last-layer reasoning features of the observer. The resulting z_t encodes the visually verified task stage, task-relevant visual evidence, and immediate objective.

Then, the cues z_t condition feature-wise linear modulation (FiLM) [50] to incorporate the stage information into the policy's vision-language context P_t. Specifically, a lightweight bottleneck mapping $h(\cdot)$ predicts feature-wise modulation parameters:

$$[\gamma_t, \beta_t] = h(z_t), \qquad \gamma_t, \beta_t \in \mathbb{R}^D. \tag{8}$$

These parameters scale and shift the context features through a residual feature-wise modulation to generate progress-aware context tokens $\tilde{P}_t$:

$$\widetilde{P}_t = \Big[P_t + \alpha\big(\gamma_t \odot P_t + \beta_t\big)\Big] \odot M_t, \tag{9}$$

where $P_t \in \mathbb{R}^{N_p \times D}$ contains N_p context tokens with feature dimension D, $M_t \in 0, 1^{N_p}$ is their validity mask, and $\alpha \in \mathbb{R}_+$ controls the modulation strength. The parameters γ_t and β_t are broadcast across tokens, and M_t across feature dimensions. With small initialization, the FiLM branch starts near identity and limits disruption to the pretrained representation. The resulting $\widetilde{P}_t$ provides a progress-aware context for subsequent attention rebalancing and action generation.

During training-data preparation, the same reasoning procedure partitions demonstrations at visually verified stage transitions and annotates observations with stage evidence, progress, immediate objectives, and cues z_t. At deployment, the frozen observer recomputes z_t from new observations, allowing feature modulation to track the visually verified task progress.

C. Stage-Aware Attention Rebalancing

The diagnosed state-dominant bias motivates adapting attention to the visually verified task stage. Building on Visual-Progress Grounding, we introduce Stage-Aware Attention Rebalancing, where the visual-progress cue z_t guides both group-wise attention modulation and its selective allocation within vision.

Specifically, the action expert takes the progress-aware vision-language context $\widetilde{P}_t = [\widetilde{V}_t; \widetilde{L}_t]$, proprioceptive state tokens s_t, and noisy action tokens x_t^τ as inputs, forming four groups $\mathcal{G} = \{\mathcal{V}, \mathcal{L}, \mathcal{S}, \mathcal{A}\}$. Conditioned on the cue z_t and execution state s_t, an MLP-based predictor ϕ produces stage-conditioned group-wise modulation signals $m_t \in \mathbb{R}^4$:

$$m_t = \phi([z_t; s_t]). \tag{10}$$

The sign of each group-wise signal specifies whether the corresponding attention logits should be increased or decreased.

Raw group-wise signals have unconstrained magnitudes, while modality groups contain different numbers of tokens. We then apply group-wise dual calibration to bound adjustment strength and regulate token-wise allocation, as illustrated in Figure 4 (b). First, *strength calibration* bounds each group-wise signal while preserving its sign. For each group $i \in \mathcal{G}$, we compute $\widetilde{m}_t^i = \tanh(m_t^i)$.

Next, *scale calibration* distributes each bounded signal across its group's tokens. For non-visual groups, we allocate the signal uniformly. For vision, task-progress evidence is often localized around the target object [51] and its interaction with the robot. We therefore use the same z_t to guide allocation within the visual group. Specifically, we project z_t into a query and match it against the modulated visual tokens:

$$q_t^{\text{vis}} = f_{\text{loc}}(z_t), \qquad \rho_{t,j} = \frac{\widetilde{v}_{t,j}^{\top} q_t^{\text{vis}}}{\|q_t^{\text{vis}}\|_2 + \epsilon}. \tag{11}$$

where $\epsilon > 0$ ensures numerical stability. We then normalize these stage-relevance scores into visual allocation weights, $w_t = \text{softmax}(\rho_t)$, where $w_{t,j}$ is the weight assigned to visual token j. For a token j in group $i \in \mathcal{G}$, the modulation is

$$\delta_{t,j} = \begin{cases} w_{t,j} \cdot \widetilde{m}_t^i, & i = \mathcal{V}, \\ \widetilde{m}_t^i / n_i, & i \in \mathcal{L} \cup \mathcal{S} \cup \mathcal{A}. \end{cases} \tag{12}$$

where n_j denotes the token count of group i. This allocation preserves $\sum_{j \in i} \delta_{t,j} = \widetilde{m}_t^i$, preventing each group's total additive logit adjustment from growing with its token count while directing visual modulation toward stage-relevant tokens.

Finally, we add the calibrated modulation signals to the action expert's attention logits to realize stage-aware rebalancing at the attention level:

$$H_t = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_h}} + \delta_t\right) V, \tag{13}$$

where Q contains the state and action queries, K and V are the key and value projections of the input tokens, and d_h is the attention-head dimension. The signals δ_t are broadcast across query positions, with the base attention mask retained. The resulting H_t integrates multimodal evidence through stage-conditioned attention and is used for subsequent action generation.

The prediction head for m_t is zero-initialized, yielding $\delta_t = 0$. Combined with the small-initialized FiLM branch, this keeps ReViP close to the pretrained policy at initialization while learning stage-conditioned modulation through the base VLA's action objective.

Training Objective. For a flow-matching base VLA, ReViP retains the original action formulation and training objective. Conditioned on the rebalanced representation H_t, the action expert is trained with:

$$\mathcal{L}_{\text{action}} = \|v_\theta(x_t^\tau, \tau \mid H_t) - u_t^\tau\|_2^2. \tag{14}$$

where x_t^τ and u_t^τ denote the noisy action and target velocity, respectively. Thus, this objective trains the visual-progress modulation to guide action generation without changing the base policy's action space or supervision, enabling corrective actions (*e.g.*, re-localizing and regrasping a dropped object) instead of following an invalidated task trajectory.

D. ReViP-Flash: In-Policy Visual-Progress Grounding

To further improve deployment efficiency, we introduce ReViP-Flash, which distills the external task-stage observer into a lightweight in-policy module, eliminating the additional VLM forward pass during execution. It learns to predict visual-progress cues from the VLA's native representations and replaces the external observer while preserving the downstream ReViP modules.

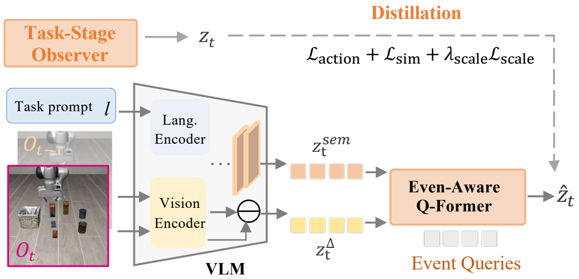


Fig. 6. **ReViP-Flash.** We replace the external observer with a lightweight event-aware in-policy observer for efficient visual-progress grounding.

As illustrated in Figure 6, ReViP-Flash constructs two complementary semantic and temporal feature streams, *i.e*, semantic tokens z_t^{sem} encode the current task-relevant scene and instruction, while temporal tokens z_t^{Δ} capture changes between consecutive replanning observations:

$$z_t^{\text{sem}} = \phi^{\text{sem}}(I_t, \ell), \qquad z_t^{\Delta} = \phi^{\text{vis}}(I_t) - \phi^{\text{vis}}(I_{t-1}), \tag{15}$$

where ϕ^{sem} and ϕ^{vis} select token streams already produced by the VLA backbone, and I_{t-1} denotes the previous replanning observation. The feature differences provide temporal evidence of events that may invalidate task progress, such as an object drop. An *event-aware Q-Former* [52] aggregates both streams using learnable event queries Q_{event}, followed by an output projection:

$$\hat{z}_t = \text{Proj}\big(\text{QFormer}\left(Q_{\text{event}}, [z_t^{\text{sem}}; z_t^{\Delta}]\right)\big) \in \mathbb{R}^d, \tag{16}$$

The predicted cue $\hat{z}_t$ thus shares the same interface as the teacher cue z_t and directly conditions feature modulation and Stage-Aware Attention Rebalancing.

To train the in-policy observer, we run the external observer offline over the training trajectories and cache its cues z_t as teacher targets. We first align the predicted and teacher cues through normalized feature matching:

$$\mathcal{L}_{\text{sim}} = \frac{1}{d} \|\mathcal{N}_\epsilon(\hat{z}_t) - \mathcal{N}_\epsilon(z_t)\|_2^2, \tag{17}$$

where $\mathcal{N}\epsilon(u) = u/(\|u\|_2 + \epsilon)$, $\epsilon > 0$ ensures numerical stability, and d is the cue dimension. This loss primarily aligns cue direction without explicitly matching raw magnitudes.

Since the unnormalized cues directly affect feature modulation and attention rebalancing, we additionally align their scales:

$$\mathcal{L}_{\text{scale}} = \left[\log(\|\hat{z}_t\|_2 + \epsilon) - \log(\|z_t\|_2 + \epsilon)\right]^2. \tag{18}$$

The complete training objective combines cue-level distillation with action supervision:

$$\mathcal{L}_{\text{Flash}} = \mathcal{L}_{\text{action}} + \mathcal{L}_{\text{sim}} + \lambda_{\text{scale}}\mathcal{L}_{\text{scale}}, \tag{19}$$

where λ_{scale} weights the scale-alignment term.

During distillation, the VLA backbone, feature-modulation module, Stage-Aware Attention Rebalancing module, and action expert remain frozen. Only the event-aware Q-Former and output projection are optimized. The frozen downstream modules remain differentiable with respect to $\hat{z}_t$, allowing $\mathcal{L}_{\text{action}}$ to propagate through the rebalancing pathway and train the observer to produce cues that support action generation,

TABLE IV

Experimental Results of False-Completion Benchmark. FCSR across eight tasks spanning visual-progress conflicts: Object Drop, Distractor Swap, and Relayout, with overall SR additionally reported. ♣ indicates non-tabletop cabinet scenes. Best and second-best results are shown in **bold** and underline, respectively, and Δ denotes the gain from integrating ReViP. ReViP outperforms VLAs and VLA+VLM/agentic systems, with consistent plug-and-play gains across heterogeneous VLA backbones.

Methods	**Object-Drop↑**					**Distractor-Swap↑**		**Relayout↑**	**Average**	
	Butter	Cheese	Bottle	Carton	Bowl♣	Butter	Cheese	Bowl-Plane	**FCSR↑**	**SR↑**
① ***VLA-Only:***										
OpenVLA [2]	12%	30%	16%	6%	2%	0%	0%	12%	10%	9%
π_0-Fast [4]	28%	62%	20%	16%	**58%**	18%	32%	84%	40%	37%
UniVLA [53]	18%	40%	16%	10%	6%	4%	12%	20%	16%	14%
Evo-1 [54]	26%	80%	38%	34%	22%	8%	14%	60%	35%	35%
StarVLA [55]	28%	84%	42%	50%	24%	24%	40%	2%	37%	36%
② ***VLA w/ External VLM:***										
π_0 w/ AHA [38]	38%	80%	38%	**74%**	28%	8%	24%	74%	46%	43%
π_0 w/ VoLoAgent [13]	38%	90%	38%	72%	24%	22%	**52%**	38%	47%	45%
π_0 w/ HarnessVLA [14]	**48%**	90%	34%	70%	36%	14%	28%	76%	50%	49%
③ ***Ours:***					***Flow-matching VLA***					
π_0 [9]	24%	78%	26%	40%	20%	6%	24%	70%	36%	35%
w/ ReViP	**48%**	**92%**	**72%**	**74%**	52%	**30%**	44%	**88%**	**63%**	**62%**
Δ	+24%	+14%	+46%	+34%	+32%	+24%	+20%	+18%	+27%	+27%
					Autoregressive VLA					
OpenVLA-OFT [12]	16%	42%	18%	12%	34%	2%	10%	18%	19%	18%
w/ ReViP	30%	74%	52%	38%	44%	14%	28%	48%	41%	38%
Δ	+14%	+32%	+34%	+26%	+10%	+12%	+18%	+30%	+22%	+20%
					Dual-system VLA					
GR00T N1.5 [10]	22%	54%	18%	22%	38%	6%	20%	34%	27%	24%
w/ ReViP	28%	70%	46%	38%	42%	20%	48%	58%	44%	41%
Δ	+6%	+16%	+28%	+16%	+4%	+14%	+28%	+24%	+17%	+17%

rather than feature matching alone. At deployment, the in-policy observer replaces the external TSO without changing the downstream ReViP pipeline, eliminating the additional VLM forward pass while retaining nearly the full robustness benefit of ReViP.

V. Experiments

We conduct extensive simulation and real-world experiments to evaluate ReViP across six complementary aspects. We first 1) assess its effectiveness on the False-Completion Benchmark (Sec. V-B) and 2) evaluate its generalization to broader manipulation settings, *i.e*, LIBERO, LIBERO-Plus, and RoboTwin 2.0 (Sec. V-C). We then 3) examine its complementarity with recovery supervision (Sec. V-D) and 4) validate key designs through ablations (Sec. V-E). Finally, we 5) evaluate its plug-and-play generality across VLA and observer architectures (Sec. V-F), and 6) assess efficiency and real-world performance (Secs. V-I and V-H).

A. Experimental Setup

Evaluation Setup. We evaluate robustness to visual-progress conflicts on our False-Completion Benchmark Suite, with 50 trials per task following the perturbation and conflict-validity protocols in Sec. III-B. We further evaluate standard manipulation on LIBERO [11], robustness under challenging variations on LIBERO-Plus [15], and bimanual manipulation on RoboTwin 2.0 [16].

Evaluation Metrics. We report *Task Success Rate* (SR) on all benchmarks and additionally report *False-Completion Success Rate* (FCSR) over valid conflict trials on the False-Completion Benchmark Suite. A rollout succeeds when the instructed task goal is reached within the prescribed step budget. All methods are evaluated under the same task configurations and evaluation conditions. Detailed protocols are provided in Sec. III of the Supplementary Material.

Baselines. We compare against representative VLAs including OpenVLA [2], OpenVLA-OFT [12], SpatialVLA [3], π_0 [9], π_0-Fast [4], $\pi_{0.5}$ [6], CoT-VLA [26], STAR [27], TriVLA [28], GR00T N1.5 [10], UniVLA [53], and StarVLA [55]. We also include recent SOTA VLA+VLM/agentic baselines, *i.e*, VoLoAgent [13] and HarnessVLA [14] to compare with external feedback and agentic recovery, alongside MIP [56] and RoboFlow4D [57] as additional manipulation baselines. For LIBERO-Plus, we also compare with WorldVLA [58], NORA [59], and VLA-Adapter [60]. For RoboTwin 2.0 [16], we further include DP3 [61], RDT [62], and π_0.

Implementation Details. Our default (**ReViP**) setting is instantiated on π_0, a representative model, as the backbone policy, with Qwen2.5-VL-3B [63] serving as the task-stage observer. Training is performed on 8 × H100 GPUs (80GB) with a total batch size of 32 for 60k training steps, using AdamW, gradient clipping at 1.0, and EMA decay of 0.999. The observer runs asynchronously once per action chunk. For a fair comparison, we reimplement VoLoAgent [13] and HarnessVLA [14] using the same π_0 backbone as ReViP.

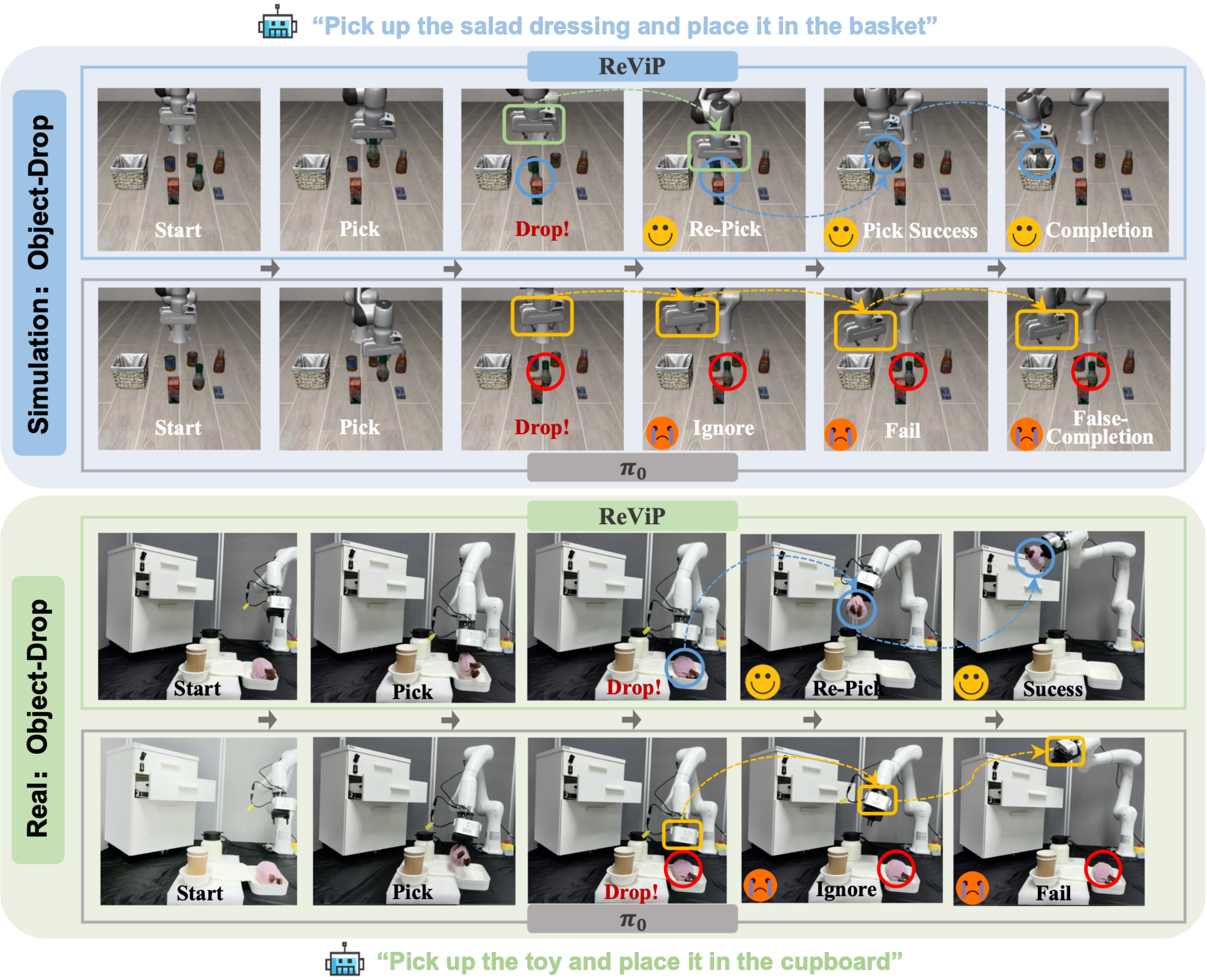


Fig. 7. **Qualitative comparisons under Object Drop in simulation (top) and the real world (bottom).** ReViP re-grounds the invalidated task progress from updated visual evidence and re-picks the target to achieve true completion ☺, whereas π_0 fails to react to the visible failure and leads to false completion ☹. Consistent behavior is observed in the real world, demonstrating how ReViP closes the perception-action loop under visual-progress conflicts.

B. Evaluation on the False-Completion Benchmark

Quantitative Results. Table IV reports FCSR and overall SR on the False-Completion Benchmark. ❶ ReViP achieves the best overall performance, with 63% average FCSR and 62% SR, outperforming the strongest VLA baseline, π_0-Fast, by 23% and 25%, respectively. Recent VLAs such as Evo-1 and StarVLA achieve 35%/35% and 37%/36% FCSR/SR, respectively, indicating that strong standard manipulation performance does not ensure robustness to visual-progress conflicts. With the same π_0 backbone, ReViP also outperforms VLA+VLM and agentic baselines, including VoLoAgent [13] (47%/45%) and HarnessVLA [14] (50%/49%). This matched-backbone comparison supports the benefit of policy-level visual-progress grounding and attention rebalancing. ❷ ReViP consistently improves performance across all three conflict families. Compared with π_0, ReViP increases average FCSR on *Object Drop* (37.6%→67.6%), on *Distractor Swap* (15%→37%), and on *Relayout* (70%→88%). These gains demonstrate its effectiveness under conflicts involving lost object possession, target re-grounding, and adaptation to changed spatial configurations. ❸ The improvements extend across heterogeneous VLA backbones. Integrating ReViP with representative flow-matching (π_0 [9]), autoregressive (OpenVLA-OFT [12]), and dual-system (GR00T N1.5 [10]) VLAs improves average FCSR by 27%, 22%, and 17%, respectively. Further evaluations across VLA backbones and task-stage observers are provided in Sec. V-F.

Qualitative Results. Figure 7 (Top) provides a representative comparison between ReViP and π_0 under Object Drop. After the bottle slips from the gripper, π_0 continues toward the destination despite updated visual observations, behaving as if the object were still held and ultimately exhibiting false completion. In contrast, ReViP responds to the updated task progress, redirects execution toward the dropped object, re-picks it, and subsequently completes the task.

This comparison illustrates how ReViP translates conflicting visual evidence into corrective actions, closing the loop between visual progress verification and ongoing execution.

C. Evaluation on Diverse Manipulation Benchmarks

Performance on LIBERO. We evaluate the LIBERO suites [11], including Spatial, Object, Goal, and Long (also called 10), each containing 10 tasks and 50 demonstrations. As shown in Table V, ReViP achieves the best average

TABLE V
EXPERIMENTAL RESULTS ON THE LIBERO BENCHMARK. SUCCESS RATES ACROSS FOUR TASK SUITES: SPATIAL, OBJECT, GOAL, AND LONG. THE BEST AND SECOND-BEST RESULTS ARE SHOWN IN **BOLD** AND UNDERLINE, RESPECTIVELY.

Methods	Spatial↑	Object↑	Goal↑	Long↑	Average SR↑
SpatialVLA [3]	88.2%	89.9%	78.6%	55.5%	78.1%
π_0 [9]	96.8%	**98.8%**	**95.8%**	85.2%	94.2%
π_0-Fast [4]	96.4%	96.8%	88.6%	60.2%	85.5%
CoT-VLA [26]	87.5%	91.6%	87.6%	69.0%	83.9%
STAR [27]	95.5%	98.3%	95.0%	88.5%	94.3%
$\pi_{0.5}$-KI [6]	98.0%	97.8%	95.6%	85.8%	94.3%
TriVLA [28]	91.2%	93.8%	89.8%	73.2%	87.0%
GR00T N1.5 [10]	92.0%	92.0%	86.0%	76.0%	86.5%
MIP [56]	97.6%	95.8%	95.2%	82.2%	92.7%
RoboFlow4D [57]	90.2%	97.0%	88.4%	75.2%	87.7%
UniVLA [53]	96.5%	96.8%	95.6%	**92.0%**	95.2%
ReViP	**99.0%** (+2.2%)	**98.8%** (+0.0%)	95.6% (-0.2%)	90.6% (+5.4%)	**96.0%** (+1.8%)

TABLE VI
EXPERIMENTAL RESULTS ON THE LIBERO-PLUS BENCHMARK. SUCCESS RATES UNDER FOUR CHALLENGING VARIATIONS: BACKGROUND, LANGUAGE, LAYOUT, AND LIGHT. THE BEST AND SECOND-BEST RESULTS ARE SHOWN IN **BOLD** AND UNDERLINE, RESPECTIVELY.

Methods	Background↑	Language↑	Layout↑	Light↑	Average SR↑
OpenVLA [2]	34.8%	23.0%	28.5%	8.1%	23.6%
WorldVLA [58]	17.1%	41.6%	38.0%	43.7%	35.1%
NORA [59]	58.6%	65.1%	62.1%	45.7%	57.9%
UniVLA [53]	81.0%	69.6%	31.9%	69.0%	62.9%
π_0-Fast [4]	73.2%	61.0%	68.8%	73.2%	69.1%
π_0 [9]	81.4%	58.8%	68.9%	85.0%	73.5%
VLA-Adapter [60]	76.6%	**73.8%**	70.2%	71.0%	72.9%
ReViP	**86.8%** (+5.4%)	65.0% (+6.2%)	**72.1%** (+3.2%)	**92.1%** (+7.1%)	**79.0%** (+5.5%)

SR of 96.0%, among the compared methods, outperforming UniVLA, STAR, and $\pi_{0.5}$. It improves over its π_0 backbone from 94.2% to 96.0%, with notable gains on Spatial (96.8% → 99.0%) and Long (85.2% → 90.6%).

These results demonstrate improved standard manipulation performance, particularly on long-horizon tasks.

Robustness on LIBERO-Plus. We further evaluate ReViP on LIBERO-Plus [15], a robustness-oriented extension of LIBERO with controlled visual, semantic, spatial, and sensor conditions. We evaluate four representative variations: *Background*, *Language*, *Layout*, and *Light*. Unlike our False-Completion Benchmark, these variations test broader distribution shifts without explicitly inducing visual-progress conflicts. As shown in Table VI, ReViP consistently improves over π_0 across all four variations, increasing mean SR from 73.5% to 79.0% The gains are particularly pronounced under background (81.4% → 86.8%), layout (68.9% → 72.1%), and lighting (85.0% → 92.1%) changes, while language robustness also improves from 58.8% to 65.0%. These consistent gains show that ReViP's benefits extend beyond the constructed conflicts to broader changes in task-relevant evidence.

Moreover, we evaluate ReViP on four contact-rich bimanual tasks from RoboTwin 2.0 [16], where it improves average SR from 10% to 20% over π_0. Detailed settings and results are provided in Sec. IV of the Supplementary Material.

D. Complementarity with Recovery Supervision.

Sec. III-C shows that recovery-data augmentation improves π_0's recovery performance at the cost of substantially degrading standard manipulation, *e.g.*, LIBERO tasks. To examine whether ReViP alleviates this trade-off, we augment both π_0 and ReViP with matched budgets of 0, 80, 160, and 240 recovery trajectories, reporting FCSR on the False-Completion Benchmark and SR on LIBERO. ❶ **ReViP mitigates false completion without sacrificing standard manipulation.** As shown in Figure 8 (a)-(d), ReViP consistently achieves higher FCSR and SR than the base π_0 policy across all recovery-data budgets. Notably, without additional recovery data, ReViP reaches 63% FCSR, matching π_0 trained with 240 additional recovery trajectories while achieving 21.4% higher LIBERO SR. This comparison demonstrates that ReViP achieves strong recovery performance without the additional data-collection cost or standard-performance degradation observed for the augmented baseline. ❷ **ReViP further benefits from recovery supervision.** Adding recovery trajectories improves ReViP's FCSR (63% → 71%) while maintaining strong LIBERO performance. In contrast, recovery gains for π_0 come with a substantial decline in standard task success.

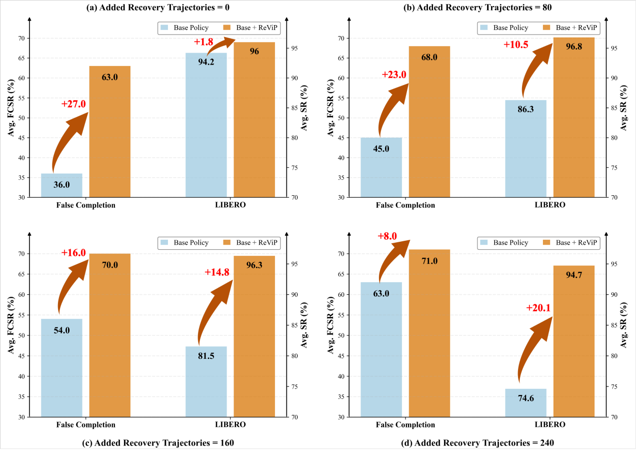


Fig. 8. **Effect of recovery-data augmentation on false-completion recovery.** ReViP unlocks the benefits of recovery-data augmentation without sacrificing standard manipulation performance.

TABLE VII
ABLATION STUDY OF REVIP ON THE FALSE-COMPLETION BENCHMARK. WE COMPARE TASK-STAGE REASONING, CUE-INJECTION STRATEGIES, AND STAGE-AWARE ATTENTION REBALANCING. THEIR FULL COMBINATION ACHIEVES THE BEST PERFORMANCE, DEMONSTRATING THE COMPLEMENTARY CONTRIBUTIONS OF THESE COMPONENTS.

#	Methods	Visual-Progress Grounding			Stage-Aware Attention Rebalancing	FC Category Avg.			Avg. FCSR ↑
		observer	task-stage reasoning	cue injection		Object Drop↑	Distractor Swap↑	Relayout↑	
1	Baseline π_0 [9]	✗	✗	–	–	38%	15%	70%	36%
2	State-free Baseline [43]	✗	✗	masking state input	–	48%	20%	78%	44% (+ 8%)
3	Append observer text to instruction	✓	✓	text-level prompt concat	–	47%	17%	72%	43% (+ 7%)
4	Append observer hidden as prefix token	✓	✓	token-level concat	–	54%	25%	74%	49% (+13%)
5	Raw VLM hidden → FiLM	✓	✗	FiLM	–	50%	23%	76%	46% (+10%)
6	Reasoning + FiLM	✓	✓	FiLM	–	58%	31%	84%	55% (+19%)
7	ReViP (Full model)	✓	✓	FiLM	✓	**68%**	**37%**	**88%**	**63%** (+27%)

TABLE VIII
ABLATION STUDY OF DUAL CALIBRATION. AVERAGE FCSR ON THE FALSE-COMPLETION BENCHMARK IS REPORTED.

Variant	Calibration			Avg. FCSR ↑
	Raw Signals	Strength Calib.	Scale Calib.	
Grounding only	✗	✗	✗	55%
w/o Calibration	✓	✗	✗	53%
+ Strength Calibration	✓	✓	✗	59%
Full ReViP	✓	✓	✓	**63%**

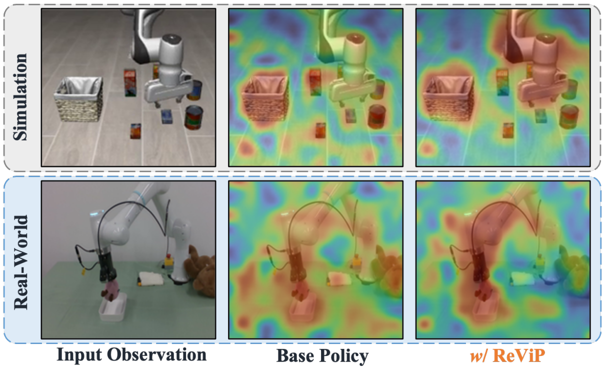


Fig. 9. **Visualization results.** Compared with the base policy, ReViP produces more concentrated visual responses around progress-relevant regions, including the robot, manipulated objects, and their interaction areas.

Overall, these results thus demonstrate the complementarity of visual-progress rebalancing and recovery supervision and validate the motivation in Sec. III-C: visual-progress grounding addresses the limitation exposed by recovery-data augmentation and provides a stronger basis for benefiting from additional corrective supervision.

E. Ablation Study of ReViP

We examine the sources of ReViP's gains through ablations on the False-Completion Benchmark and report average FCSR.

Analysis of Visual-Progress Rebalancing. We evaluate the contributions of the core designs of ReViP, *i.e*, the cue extraction and integration within *Visual-Progress Grounding* and the additional contribution of *Stage-Aware Attention Rebalancing*. We thus organize the ablation study through three controlled comparisons (Table VII): **Q1:** *What makes visual-progress cues effective: generic VLM representations or task-stage reasoning?* **Q2:** *Is feature modulation more effective than direct cue injection?* **Q3:** *Does stage-aware attention rebalancing provide additional gains beyond representation-level grounding?*

(1) Task-stage reasoning provides effective visual-progress cues. To answer Q1, conditioning FiLM on raw VLM hidden representations yields 46% FCSR (Row 5), whereas task-stage reasoning raises it to 55% (Row 6). Meanwhile, scaling the observer from 3B to 72B provides only a modest gain (Table IX). These results suggest that the effectiveness of visual-progress cues relies more on task-stage reasoning than simply increasing observer capacity, highlighting the importance of converting generic visual representations into task-relevant progress information. **(2) Feature modulation effectively improves cue integration.** To answer Q2, we compare FiLM with cue injection through the language instruction and policy prefix. These alternatives achieve 43% and 49% FCSR, respectively (Rows 3–4), compared with 55% for FiLM (Row 6). Thus, the results show that how the cues enter the policy matters: feature modulation incorporates visual-progress cues more effectively than simply appending them to the policy inputs. **(3) Stage-Aware Attention Rebalancing provides complementary gains.** To answer Q3, adding attention rebalancing to the grounded representation in Row 6 further improves average FCSR from 55% to 63%. The gains are consistent across Object Drop (58%→68%), Distractor Swap (31%→37%), and Relayout (84%→88%). Importantly, this additional gain of +8% increases the overall improvement over π_0 from +19% to +27%, demonstrating that attention-level rebalancing complements representation-level grounding in mitigating false completion.

Effectiveness of Group-wise Dual Calibration. We further examine the contribution of dual calibration to stage-aware attention rebalancing. As shown in Table VIII, directly applying uncalibrated modulation signals reduces FCSR from 55% to 53%. Introducing strength calibration raises FCSR to 59%, while adding scale calibration further improves it to 63%. These results support calibrating both modulation strength and token-wise allocation to make the predicted group-wise signals effective for attention rebalancing.

Heatmap Visualization. To further examine whether ReViP effectively focuses the policy's visual responses on stage-relevant evidence, we compare it and π_0 in Figure 9. While π_0 exhibits diffuse responses across the scene, ReViP concentrates its responses around progress-relevant regions, including the robot, manipulated objects, and their interaction areas. This pattern suggests greater emphasis on visual evidence relevant to task progress and is also observed in the real-world examples (bottom). Quantitative real-world evaluation is presented in Sec. V-H, and details of the visualization procedure and additional results are provided in Supplementary Material.

TABLE IX
GENERALITY ACROSS TASK-STAGE OBSERVERS. WE REPORT FCSR WITH DIFFERENT VLM BACKBONES FOR OBSERVERS.

Observer Backbone	FC Category Avg.			Avg. FCSR ↑
	Object Drop ↑	Distractor Swap ↑	Relayout ↑	
w/o TSO (Baseline) [9]	38%	15%	70%	36%
LLaVA-1.5-7B [64]	60%	27%	74%	54% (+18%)
InternVL2.5-8B [65]	65%	25%	80%	57% (+21%)
Qwen2.5-VL-3B [63]	68%	37%	**88%**	63% (+27%)
Qwen2.5-VL-72B [63]	**69%**	**39%**	**88%**	**64%** (+28%)

TABLE X
ABLATION STUDY OF REVIP-FLASH DISTILLATION. AVERAGE FCSR ON THE FALSE-COMPLETION BENCHMARK IS REPORTED.

Distillation Variant	$\mathcal{L}_{\text{sim}}$	$\mathcal{L}_{\text{scale}}$	$\mathcal{L}_{\text{action}}$	Avg. FCSR ↑
w/ Directional Cue Alignment	✓	✗	✗	56%
+ Cue-Scale Alignment	✓	✓	✗	58%
ReViP-Flash (+ Action Supervision)	✓	✓	✓	**62%**
Full ReViP (Teacher)	–	–	–	63%

F. Plug-and-Play Generality of ReViP

To assess ReViP's modularity, we evaluate its applicability across VLA backbones and task-stage observers.

Plug-and-play across VLA architectures. ReViP introduces less than **0.4%** additional trainable parameters without requiring redesign of the base policy's backbone or training objective. As shown in Table IV, the same ReViP design consistently improves performance across π_0, OpenVLA-OFT, and GR00T N1.5 (Table IV), demonstrating its applicability across different VLA architectures.

Generality across task-stage observers. We instantiate observer with different VLM families and scales, including LLaVA-1.5-7B [64], InternVL2.5-8B [65], and Qwen2.5-VL [63]. All observer variants improve over the base policy, achieving 54%–64% average FCSR (Table IX). Notably, Qwen2.5-VL-3B achieves 63% FCSR, close to the 64% obtained with Qwen2.5-VL-72B, indicating that ReViP benefits more from through task-stage reasoning than simply scaling observer capacity.

Furthermore, we evaluate ReViP under controlled stage and target corruption, demonstrating its tolerance to perturbations in visual-progress cues (Section IV of the Supplementary Material).

G. Ablation Study of ReViP-Flash

To examine how cue alignment and action supervision contribute to distillation, we ablate the training objectives of ReViP-Flash in Table X. Directional cue alignment $\mathcal{L}_{\text{sim}}$ alone achieves 56% FCSR, and adding scale alignment $\mathcal{L}_{\text{scale}}$ improves it to 58%. Action supervision further raises FCSR to 62%, approaching the 63% achieved by ReViP with the external observer. These results support combining cue-level alignment with downstream action supervision to distill visual-progress cues that are both consistent with the teacher and useful for action generation. Additional sensitivity analysis of λ_{scale} is provided in Sec. IV of the Supplementary Material.

H. Real-World Evaluation

We further evaluate whether ReViP's gains extend to physical manipulation, examining recovery from visual-progress conflicts and robustness to challenging scene and sensing conditions.

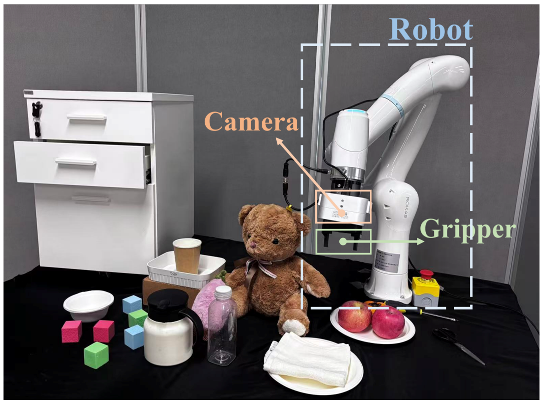


Fig. 10. Real-world experiments use a robotic arm with grippers and cameras.

Robot platform. As shown in Figure 10, experiments use a 6-DoF ROKAE robotic arm with a JODELL gripper and two RGB-D cameras providing wrist-mounted and third-person views. The policy receives these observations and a 7-D proprioceptive state comprising six joint positions and the gripper state, and predicts actions in chunks of 50 steps.

Baselines. We compare ReViP and ReViP-Flash with two representative VLAs, *i.e*, flow matching-based π_0 [9] and autoregressive-based π_0-Fast [4]. All methods use the same demonstrations and evaluation protocol, and each task is evaluated over 10 trials.

Tasks and Evaluation Settings. We evaluate two representative types of manipulation tasks: (1) *Object Placement*, which requires grasping a target object and placing it into a designated container, and (2) *Drawer Manipulation*, a multi-stage task involving object placement followed by drawer interaction. We collect 50-100 demonstrations per task and evaluate under two settings. The *Controlled FC Setting* introduces Drop and Relayout perturbations to directly test recovery under physical visual-progress conflicts. The *Challenging Real-World Setting* further varies scene appearance, sensing conditions, target scale, distractor clutter, and multi-stage task complexity to assess broader robustness. Detailed task definitions, hardware variations, and perturbation protocols are provided in the Supplementary Material.

Real-World Results. Table XI shows that ReViP significantly outperforms both π_0 and π_0-Fast across both real-world settings. (1) Under the controlled FC setting, ReViP achieves 88.0% average SR, improving its π_0 backbone by 26.0%. For Object Drop, successful trials increase from 4/10 to 8/10 on Object Placement and from 3/10 to 7/10 on Drawer Manipulation. Figure 7 (bottom) illustrates how ReViP regrounds the invalidated task progress, re-localizes the dropped target, and revises its subsequent actions instead of continuing toward false completion. (2) Under the challenging real-world setting, ReViP achieves 73.3% average SR, compared with 36.7% for π_0 and 23.3% for π_0-Fast. Improvements across distractor-heavy scenes, small targets, object drops, and cluttered multi-stage tasks demonstrate its effectiveness in challenging physical environments (Figure 1, bottom). It

TABLE XI

REAL-WORLD EVALUATION UNDER CONTROLLED FALSE-COMPLETION AND CHALLENGING CONDITIONS. THE CHALLENGING SETTING FURTHER VARIES SENSING, SCENE COMPLEXITY, TARGET SCALE, AND TASK HORIZON. SUCCESS RATES (SR) ARE REPORTED OVER 10 TRIALS PER CONDITION. THE BEST RESULT IS SHOWN IN **BOLD**. REVIP-FLASH DENOTES THE LIGHTWEIGHT DISTILLED VARIANT, AND Δ DENOTES THE PERFORMANCE GAIN OF FULL REVIP OVER ITS π_0 BACKBONE.

Settings	Tasks	Conditions	π_0 [9]	π_0-Fast [4]	ReViP-Flash	π_0+ReViP	Δ
Controlled FC Real-World Setting	**Object Placement**	Cup → Box	8/10	7/10	**10/10**	**10/10**	+20%
		+ Drop	4/10	3/10	7/10	**8/10**	+40%
	Drawer Manipulation	Toy → Drawer	7/10	7/10	**10/10**	9/10	+20%
		+ Close	9/10	8/10	**10/10**	**10/10**	+10%
		+ Drop	3/10	3/10	**7/10**	**7/10**	+40%
		Setting Avg. SR	62.0%	56.0%	**88.0%**	**88.0%**	**+26.0%**
Challenging Real-World Setting	**Object Placement**	Cup → Box	5/10	3/10	**10/10**	9/10	+40%
		+ Drop	4/10	1/10	7/10	**8/10**	+40%
		+ Distractor	3/10	2/10	**8/10**	6/10	+30%
		Small Cube → Box	4/10	2/10	**8/10**	7/10	+30%
		+ Distractor	2/10	0/10	**7/10**	6/10	+40%
	Drawer Manipulation	Toy → Drawer → Close	4/10	6/10	**8/10**	**8/10**	+40%
		Setting Avg. SR	36.7%	23.3%	**80.0%**	73.3%	**+36.6%**
Overall			48.2%	38.2%	**83.6%**	80.0%	**+31.8%**

TABLE XII

INFERENCE LATENCY AND COMPUTATIONAL OVERHEAD OF REVIP.

Component	π_0 [9]	π_0+ReViP	ReViP-Flash
Shared VLM Forward	$\sim$30 ms	$\sim$30 ms	$\sim$31 ms
Observer Overhead	–	$\sim$17 ms	–
FiLM	–	$<$ 0.1 ms	$<$ 0.1 ms
Flow Matching	$\sim$15 ms	$\sim$15 ms	$\sim$15 ms
Total Latency ↓	$\sim$45 ms	$\sim$62 ms	$\sim$46 ms
Frequency ↑	$\sim$22 Hz	$\sim$16 Hz	$\sim$22 Hz
FCSR ↑	36%	**63%** (+27)	**62%** (+26)

is noted that ReViP-Flash also maintains strong real-world performance, achieving 88.0% SR in the controlled setting and 80.0% SR in the challenging setting with its lightweight in-policy observer.

I. Comparison of Efficiency and Performance

We finally evaluate the deployment efficiency and performance of ReViP. As shown in Table XII, all methods are profiled on our False-Completion Benchmark using a single NVIDIA H100 GPU, with latency averaged over 50 runs after 30 warm-up iterations. Full ReViP improves FCSR by 27% with only modest inference overhead over the π_0 backbone. Asynchronous task-stage reasoning runs once per action chunk, allowing ReViP to maintain approximately 16 Hz control. The detailed inference pipeline is provided in Sec. II of the Supplementary Material.

By replacing the external observer with a lightweight in-policy module, ReViP-Flash achieves near-backbone inference efficiency ($\sim$22 Hz) while retaining a 26% FCSR gain. This reduces deployment overhead while preserving most of ReViP's robustness improvement.

VI. CONCLUSION

We systematically study *false completion* in VLAs, a closed-loop failure in which policies continue according to presumed task progress despite conflicting visual evidence. We first introduce the False-Completion Benchmark Suite to evaluate this failure under controlled visual-progress conflicts. Our data-to-model diagnostics then suggest a *state-dominant bias in progress estimation*, highlighting the reliable execution requires to continually ground presumed progress in current visual observations and use this grounding to guide action revision. Motivated by this insight, we propose ReViP combines *Visual-Progress Grounding*, which incorporates visually verified stage cues into policy representations, with *Stage-Aware Attention Rebalancing*, which adaptively regulates multimodal attention during action generation. Extensive simulation and real-world experiments demonstrate consistent improvements over strong VLAs and VLA+VLM/agentic systems, broad generalization, and plug-and-play applicability, while ReViP-Flash retains the full robustness benefit with efficient inference. Overall, this work highlights the importance of connecting visual progress verification to action revision for reliable closed-loop VLA manipulation.

REFERENCES

[1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. T. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, "RT-2: vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*, 2023.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Conference on Robot Learning*, 2024.

[3] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang *et al.*, "Spatialvla: Exploring spatial representations for visual-language-action model," in *Robotics: Science and Systems*, 2025.

[4] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "Fast: Efficient action tokenization for vision-language-action models," in *Robotics: Science and Systems*, 2025.

[5] Z. Hou, T. Zhang, Y. Xiong, H. Duan, H. Pu, R. Tong, C. Zhao, X. Zhu, Y. Qiao, J. Dai, and Y. Chen, "Dita: Scaling diffusion transformer for generalist vision-language-action policy," in *International Conference on Computer Vision*, 2025.

[6] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Conference on Robot Learning*, 2025.

[7] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang, "Thinkact: Vision-language-action reasoning via reinforced visual latent planning," in *arXiv preprint arXiv:2507.16815*, 2025.

[8] Z. Zhong, H. Yan, J. Li, X. Liu, X. Gong, T. Zhang, W. Song, J. Chen, X. Zheng, H. Wang, and H. Li, "Flowvla: Visual chain of thought-based motion reasoning for vision-language-action models," in *arXiv preprint arXiv:2508.18269*, 2025.

[9] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," *Robotics: Science and Systems*, 2025.

[10] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[11] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.

[12] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025.

[13] S. Chen, H. Hadfield, A. Zook, M. A. Uy, C. H. Song, E. Coumans, X. Yang, F. Ladhak, Q. Qu, S. Birchfield *et al.*, "Volo: A physical orchestrator for open-vocabulary long-horizon manipulation," *arXiv preprint arXiv:2606.07723*, 2026.

[14] Y. Zhang, H. Zhang, F. Gao, X. Li, Z. Liu, C. Zhu, J. Qiu, Y. Yan, J. Liu, W. Tang *et al.*, "Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents," *arXiv preprint arXiv:2607.08448*, 2026.

[15] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei *et al.*, "Libero-plus: In-depth robustness analysis of vision-language-action models," *arXiv preprint arXiv:2510.13626*, 2025.

[16] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, "Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[17] S. Yang, H. Li, Y. Chen, B. Wang, Y. Tian, T. Wang, H. Wang, F. Zhao, Y. Liao, and J. Pang, "Instructvla: Vision-language-action instruction tuning from understanding to manipulation," in *International Conference on Learning Representations*, 2026.

[18] X. Zhou, X. Han, F. Yang, Y. Ma, and A. C. Knoll, "Opendrivevla: Towards end-to-end autonomous driving with large vision language action model," in *The Association for the Advancement of Artificial Intelligence*, 2026.

[19] H. Li, S. Yang, Y. Chen, Y. Tian, X. Yang, X. Chen, H. Wang, T. Wang, F. Zhao, D. Lin *et al.*, "Cronusvla: Transferring latent motion across time for multi-frame prediction in manipulation," in *The Association for the Advancement of Artificial Intelligence*, 2026.

[20] W. Zhang, H. Liu, Z. Qi, Y. Wang, X. Yu, J. Zhang, R. Dong, J. He, H. Wang, Z. Zhang, Y. Li, W. Zeng, and X. Jin, "DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge," in *Conference on Neural Information Processing Systems*, 2025.

[21] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. S. Ryoo, G. Salazar, P. R. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. T. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, "Rt-1: Robotics transformer for real-world control at scale," *Robotics: Science and Systems*, 2023.

[22] S. Belkhale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh, "Rt-h: Action hierarchies using language," in *Robotics: Science and Systems*, 2024.

[23] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration0," in *International Conference on Robotics and Automation*, 2024.

[24] Alexander, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," in *robotics: science and systems*, 2024.

[25] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations*, 2023.

[26] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, A. Handa, T. Lin, G. Wetzstein, M. Liu, and D. Xiang, "Cot-vla: Visual chain-of-thought reasoning for vision-language-action models," in *Conference on Computer Vision and Pattern Recognition*, 2025.

[27] H. Li, Q. Lv, R. Shao, X. Deng, Y. Li, J. HAO, and L. Nie, "Star: Learning diverse robot skill abstractions through rotation-augmented vector quantization," in *Forty-second International Conference on Machine Learning*, 2025.

[28] Z. Liu, Y. Gu, S. Zheng, Y. Fu, X. Xue, and Y.-G. Jiang, "Trivla: A triple-system-based unified vision-language-action model with episodic world modeling for general robot control," *arXiv preprint arXiv:2507.01424*, 2025.

[29] W. Li, R. Zhang, R. Shao, J. He, and L. Nie, "Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification," in *Advances in neural information processing systems*, 2025.

[30] M. H. Khan, S. Asfaw, D. Iarchuk, M. A. Cabrera, L. Moreno, I. Tokmurziyev, and D. Tsetserukou, "Shake-vla: Vision-language-action model-based system for bimanual robotic manipulations and liquid mixing," in *arXiv preprint arXiv:2501.06919*, 2025.

[31] Z. Zhou, Y. Zhu, J. Wen, C. Shen, and Y. Xu, "Chatvla-2: Vision-language-action model with open-world embodied reasoning from pre-trained knowledge," in *Advances in Neural Information Processing Systems*, 2025.

[32] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar, "Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting," *arXiv preprint arXiv:2509.22195*, 2025.

[33] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang *et al.*, "Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.

[34] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," in *International Conference on Robot Learning*, 2023.

[35] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *International Conference on Robot Learning*, 2023.

[36] Y.-X. He, L. Wei, H. Cen, J.-J. Jiang, Z. Li, G. Lu, Y. Yang, D. Zhang, and W.-S. Zheng, "A closed-loop multi-agent framework for robust multi-robot manipulation," in *robotics: science and systems*, 2026.

[37] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. de Freitas, and S. Cabi, "Vision-language models as success detectors," in *Conference on Lifelong Learning Agents*, 2023.

[38] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *International Conference on Learning Representations*, 2025.

[39] Z. Wang, Z. Zhou, J. Song, Y. Huang, Z. Shu, and L. Ma, "Vlatest: Testing and evaluating vision-language-action models for robotic manipulation," *Proceedings of ACM Software Engineering*, 2025.

[40] C. Ma, G. Yang, K. Lu, S. Xu, B. Byrne, N. Trigoni, and A. Markham, "Cyclevla: Proactive self-correcting vision-language-action models via subtask backtracking and minimum bayes risk decoding," *arXiv preprint arXiv:2601.02295*, 2026.

[41] H. Yan, Q. Li, J. Yang, and Y. Mu, "Progressvla: Progress-guided diffusion policy for vision-language robotic manipulation," *arXiv preprint arXiv:2603.27670*, 2026.

[42] S. Kim, J. Choi, S. Baek, and J.-M. Renders, "Progvla: Progress-aware robot manipulation skill learning," *arXiv preprint arXiv:2605.28231*, 2026.

[43] J. Zhao, W. Lu, D. Zhang, Y. Liu, Y. Liang, T. Zhang, Y. Cao, J. Xie, Y. Hu, S. Wang, J. Guo, D. Wang, and Y. Gao, "Do you need proprioceptive states in visuomotor policies?" *arXiv preprint arXiv:2509.18644*, 2025.

[44] J. Lu, W. Xia, Y. Wu, Z. Lu, and D. Hu, "When would vision-proprioception policies fail in robotic manipulation?" in *International Conference on Learning Representations*, 2026.

[45] J. Wang, S. Chen, Y. Cai, Y. Lu, T. Zhang, H. Shi, K. Zhang, X. Lu, H. Li, L. Chen *et al.*, "Think proprioceptively: Embodied visual reasoning through multimodal chain-of-thought for vision-language-action manipulation," *arXiv preprint arXiv:2601.07365*, 2026.

[46] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, "Robocasa: Large-scale simulation of everyday tasks for generalist robots," *arXiv preprint arXiv:2406.02523*, 2024.

[47] E. Tang, B. Yang, and X. Song, "Understanding LLM embeddings for regression," *Transactions on Machine Learning Research*, 2025.

[48] R. Meng, Z. Jiang, Y. Liu, M. Su, X. Yang, Y. Fu, C. Qin, Z. Chen, R. Xu, C. Xiong, Y. Zhou, W. Chen, and S. Yavuz, "Vlm2vec-v2: Advancing multimodal embedding for videos, images, and visual documents," *arXiv preprint arXiv:2507.04590*, 2025.

[49] Z. Li, W. Chen, J. Yu, and Z. Lu, "Functional consistency of llm code embeddings: A self-evolving data synthesis framework for benchmarking," *Expert Systems with Applications*, p. 129523, 2025.

[50] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[51] S. Kang, J. Kim, J. Kim, and S. J. Hwang, "See what you are told: Visual attention sink in large multimodal models," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 87 676–87 703.

[52] J. Li, D. Li, S. Savarese, and S. Hoi, "Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *International conference on machine learning*. PMLR, 2023, pp. 19 730–19 742.

[53] Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li, "Univla: Learning to act anywhere with task-centric latent actions," in *Robotics: Science and Systems*, 2025.

[54] T. Lin, Y. Zhong, Y. Du, J. Zhang, J. Liu, Y. Chen, E. Gu, Z. Liu, H. Cai, Y. Zou, L. Zou, Z. Zhou, G. Li, and B. Zhao, "Evo-1: Lightweight vision-language-action model with preserved semantic alignment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 13 397–13 406.

[55] S. Community, "Starvla: A lego-like codebase for vision-language-action model developing," *arXiv preprint arXiv:2604.05014*, 2026.

[56] C. Pan, G. Anantharaman, N.-C. Huang, C. Jin, D. Pfrommer, C. Yuan, F. Permenter, G. Qu, N. Boffi, G. Shi, and M. Simchowitz, "Much ado about noising: Dispelling the myths of generative robotic control," in *International Conference on Learning Representations*, 2026.

[57] S. Lin, J. Chen, H. Xu, Z. Li, G. Wang, Y. Jing, S. Xu, R. Zhao, B. Sheil, L.-P. Chau, and G. Liu, "Roboflow4d: A lightweight flow world model toward real-time flow-guided robotic manipulation," in *Forty-third International Conference on Machine Learning*, 2026.

[58] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang *et al.*, "Worldvla: Towards autoregressive action world model," *arXiv preprint arXiv:2506.21539*, 2025.

[59] C.-Y. Hung, Q. Sun, P. Hong, A. Zadeh, C. Li, U. Tan, N. Majumder, S. Poria *et al.*, "Nora: A small open-sourced generalist vision language action model for embodied tasks," *arXiv preprint arXiv:2504.19854*, 2025.

[60] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou *et al.*, "Vla-adapter: An effective paradigm for tiny-scale vision-language-action model," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 40, no. 22, 2026, pp. 18 638–18 646.

[61] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3d diffusion policy," *arXiv preprint arXiv:2403.03954*, 2024.

[62] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1b: a diffusion foundation model for bimanual manipulation," in *The Thirteenth International Conference on Learning Representations*, 2025.

[63] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin, "Qwen2.5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025.

[64] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems*, 2023.

[65] Z. Chen, J. Wu, W. Wang, W. Su, G. Chen, S. Xing, M. Zhong, Q. Zhang, X. Zhu, L. Lu *et al.*, "Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks," in *Conference on Computer Vision and Pattern Recognition*, 2024, pp. 24 185–24 198.